



6. Python: Built-in Methods

"You do not just write code. Sometimes, you just need to use what the programming language gives you"

Previous Section:

[`</>` 5. Python: Operators](#)

Contents:

[1. Common Built-in Methods](#)

[2. Data Type Conversion Methods](#)

[3. Advanced Methods](#)

[4. Bytes-Coding Methods](#)

[5. Attributive Methods](#)

[Summary](#)

[References](#)

Python, like many programming languages, comes with something incredibly powerful built directly into the language itself: **built-in methods** (or we commonly call it **built-in functions**).

You can think of them as ready-made mini programs that Python already knows how to execute. Instead of writing complicated logic from scratch every time, we simply call these functions and let Python handle the work for us.

And honestly, this is one of the reasons Python feels so beginner-friendly.

Need to print something? → `print()`

Need the length of a list? → `len()`

Need to convert text into a number? → `int()`

Simple. Direct. Clean. These built-in functions are designed to solve common problems that programmers face all the time. Some are simple utility tools, while others are powerful enough to control files, manipulate data, or even execute code dynamically.

Now technically, Python provides a *huge* collection of built-in functions. Here are all of them:

```
abs(), all(), any(), bin(), bool(), dict(), enumerate(), filter(), float(),  
format(), help(), input(), int(), len(), list(), map(), max(), min(), open(),  
pow(), print(), range(), round(), set(), sorted(), str(), sum(), tuple(), type(),  
zip(), and many more.
```

At first glance, this might feel overwhelming. But don't worry. You are not expected to memorize everything.

In this section, we'll only focus on the most important and commonly used built-in functions—the ones you'll constantly encounter while writing Python programs. The remaining functions will be introduced naturally later in the course, alongside the concepts where they actually make sense. Because learning Python is not about memorizing every tool immediately. It's about understanding *when* and *why* to use them.

1. Common Built-in Methods

These are the built-in functions you will probably use the most in everyday Python programming.

Some help us display output, some receive input from users, while others help analyze or manipulate data quickly.

The most famous ones are definitely:

- `print()` → Displays output to the console
- `input()` → Receives input from the user

But Python also provides powerful shortcuts for handling multiple values and quick calculations.

- `eval()` → Evaluates a string as real Python code
- `map()` → Applies a function to every item in an iterable
- `.split()` → Splits a string into multiple parts

And then come the "daily-use utility tools":

- `min()` → Returns the smallest value
- `max()` → Returns the largest value
- `round()` → Rounds a number
- `len()` → Returns the length of an iterable
- `sum()` → Calculates the total sum
- `sorted()` → Returns a sorted version of data

These methods save an enormous amount of time. Instead of manually writing loops and conditions for everything, Python already provides optimized tools for us.

Example:

```
# print() and input()
name = input("Enter your name: ")
print("Hello", name)
```

Enter your name: Alice
Hello Alice

```
# eval() with split() and map()
numbers = list(map(int, input("Enter numbers: ").split(
    ",")))

print("Numbers:", numbers)

# min(), max(), sum(), len()
print("Minimum:", min(numbers))
print("Maximum:", max(numbers))
print("Total:", sum(numbers))
print("Length:", len(numbers))

# sorted()
print("Sorted:", sorted(numbers))
```

Enter numbers: 12,13,15,11,23,9,12
 Numbers: [12, 13, 15, 11, 23, 9, 12]
 Minimum: 9
 Maximum: 23
 Total: 95
 Length: 7
 Sorted: [9, 11, 12, 12, 13, 15, 23]

```
# round()
pi = 3.1415926535
print("Rounded:", round(pi, 3))
```

Rounded: 3.142

2. Data Type Conversion Methods

One of the most important ideas in Python is that data can change forms. A number can become text. Text can become a number. Lists can become tuples. Tuples can become sets. These are called **type conversion functions**. Python gives us many built-in tools for this:

- `type()` → Checks the data type
- `str()` → Converts data into string
- `float()` → Converts data into decimal number

- `int()` → Converts data into integer
- `bool()` → Converts data into boolean (`True` or `False`)
- `list()` → Converts iterable into list
- `tuple()` → Converts iterable into tuple
- `set()` → Converts iterable into set
- `dict()` → Creates a dictionary
- `reversed()` → Reverses an iterable
- `filter()` → Filters elements based on a condition

These functions are extremely important because Python is heavily based on data manipulation.

Example:

```
# Original data
x = "25"
# type()
print(type(x))
# str(), int(), float(), bool()
a = int(x)
b = float(x)
c = str(a)
d = bool(a)

print("Integer:", a)
print("Float:", b)
print("String:", c)
print("Boolean:", d)
```

```
<class 'str'>
Integer: 25
Float: 25.0
String: 25
Boolean: True
```

```
# list(), tuple(), set()
data = [1, 2, 2, 3, 4]

print("List:", list(data))
print("Tuple:", tuple(data))
print("Set:", set(data))
```

List: [1, 2, 2, 3, 4]
 Tuple: (1, 2, 2, 3, 4)
 Set: {1, 2, 3, 4}

```
# dict()
pairs = [("name", "Alex"), ("age", 20)]
print("Dictionary:", dict(pairs))
```

Dictionary: {'name': 'Alex', 'age': 20}

```
# reversed()
text = "Python"
print("Reversed List:", list(reversed(text)))
```

Reversed List: ['n', 'o', 'h', 't', 'y', 'P']

```
# filter()
numbers = [1, 2, 3, 4, 5, 6]

even = list(filter(lambda x: x % 2 == 0, numbers))
print("Filter List:", even)
```

Filter List: [2, 4, 6]

3. Advanced Methods

Now we move into some methods that feel a little more “computer-science-like”. These functions are not always used every day, but they become incredibly useful once we start dealing with encoding, logic checking, memory understanding, and mathematical operations.

- `ord()` → Converts a character into its ASCII/Unicode number
- `chr()` → Converts a number into a character
- `ascii()` → Returns ASCII-safe representation of text
- `any()` → Returns `True` if at least one value is `True`
- `all()` → Returns `True` only if all values are `True`
- `id()` → Returns the memory identity of an object
- `divmod()` → Returns quotient and remainder together

These functions reveal how Python interacts internally with data and logic.

Example:

```
# ord() and chr()
print(ord('A'))
print(chr(66))
```

65

B

```
# ascii()
text = "Pythön"
print(ascii(text))
```

'Pyth\x66n'

```
# any() and all()
conditions = [True, False, True]

print(any(conditions))
print(all(conditions))
```

True

False

```
# id()
x = [1, 2, 3]
print(id(x))
```

133977865927040

```
# divmod()
result = divmod(17, 5)

print("Quotient:", result[0])
print("Remainder:", result[1])
```

Quotient: 3

Remainder: 2

4. Bytes-Coding Methods

Computers do not truly understand decimal numbers the way humans do. Internally, everything eventually becomes binary data. Python provides built-in functions to convert numbers into different numeral systems:

- `bin()` → Converts number into binary
- `hex()` → Converts number into hexadecimal
- `oct()` → Converts number into octal

These become extremely useful in areas like computer architecture, cybersecurity, or networking

Example:

```
number = 25

# bin()
print("Binary:", bin(number))
# hex()
print("Hexadecimal:", hex(number))
# oct()
print("Octal:", oct(number))
```

Binary: 0b11001

Hexadecimal: 0x19

Octal: 0o31

5. Attributive Methods

Now we enter something slightly different. Technically, these are not standalone built-in functions. They are methods attached directly to objects.

That's why we write them like this: `object.method()` instead of: `method(object)`

Examples include:

- `.split()` → Splits strings into parts
- `.sort()` → Sorts a list directly
- `.pop(i)` → Removes an item at index
- `.upper()` → Converts text to uppercase
- `.lower()` → Converts text to lowercase
- `.reverse()` → Reverses a list directly
- `.replace(old, new)` → Replaces text inside a string

Some of these behave similarly to built-in functions. For example:

- `sorted(list)` creates a new sorted list
- `.sort()` modifies the original list itself

So while they may appear similar, internally, they work differently. These methods will become much more important later when we dive deeper into: Lists; Tuples; Strings; Iterable sequences

Example:

```
# .split()
text = "Python is fun"
words = text.split()
print(words)
```

['Python', 'is', 'fun']

```
# .sort()
numbers = [5, 1, 4, 2, 3]
numbers.sort()
print("Sorted:", numbers)
# .pop(i)
numbers.pop(2)
print("Pop 2:", numbers)
# .reverse()
numbers.reverse()
print("Reversed:", numbers)
```

Sorted: [1, 2, 3, 4, 5]

Pop 2: [1, 2, 4, 5]

Reversed: [5, 4, 2, 1]

```
# .upper() and .lower()
name = "Iris"

print(name.upper())
print(name.lower())
```

IRIS

iris

```
# .replace(old, new)
sentence = "Python is hard"
sentence = sentence.replace("hard", "powerful")

print(sentence)
```

Python is powerful

Summary

Built-in functions are one of the reasons Python feels so powerful and beginner-friendly. Instead of writing everything manually, Python already provides many ready-made tools for handling calculations, inputs, outputs, type conversions, sorting, filtering, encoding, and data manipulation. In this section, we explored some of the most important ones—from simple functions like `print()`, `input()`, `len()`, and `sum()`, to more advanced tools like `map()`, `filter()`, `divmod()`, `ord()`, and `chr()`. We also introduced object-based methods such as `.split()`, `.sort()`, `.upper()`, and `.replace()`, which behave differently from normal built-in functions because they belong directly to specific objects like strings and lists.

But remember something important: you are not supposed to memorize every single method immediately. Even experienced programmers constantly search for methods and reread documentation. The goal right now is simply to understand that these tools exist—and slowly become familiar with how and when to use them. If you want more methods, they will be introduced throughout this course naturally as we learn new concepts. You can also reference the internet (there are countless sources out there). And at the end of this course, I will also create a complete cheatsheet covering everything in one place. So stay tuned for that.

References

<https://docs.python.org/3/library/functions.html>

<https://www.geeksforgeeks.org/python/python-built-in-functions/>

Next Section:

[`</>` 7. Python: Loops](#)